МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

Кафедра «Интеллектуальные информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №2

по дисциплине «ОСНОВЫ МАШИННОГО ОБУЧЕНИЯ» за 5 семестр

Тема: «Линейные модели для задач регрессии и классификации»

Выполнил:

студент 3 курса

группы АС-67

Иванович Д. И.

Проверил:

Крощенко А. А.

2026

Цель: изучить применение линейной и логистической регрессии для решения практических задач. Научиться обучать модели, оценивать их качество с помощью соответствующих метрик и интерпретировать результаты.

Общее задание: выполнить задания по варианту (регрессия и классификация), построить все требуемые визуализации и рассчитать метрики, **написать отчет, создать пул-реквест в репозиторий с кодом решения и отчетом в формате pdf.**

Вариант 3

• **Регрессия (Прогнозирование расхода топлива)**

1. Auto MPG
2. Предсказать расход топлива (mpg)
3. **Задания:**
   - загрузите данные, обработайте категориальные признаки; пропуски и категориальные признаки;
   - обучите модель линейной регрессии, используя в качестве признаков cylinders, horsepower, weight;
   - рассчитайте MSE и R^2;
   - визуализируйте зависимость mpg от horsepower с линией регрессии.

• **Классификация (Диагностика диабета)**

1. Pima Indians Diabetes
2. Предсказать наличие диабета (Outcome)
3. **Задания:**
   - загрузите данные, выполните стандартизацию признаков используйте Length1, Length2, Length3, Height, Width;
   - обучите модель логистической регрессии;
   - оцените качество, рассчитав R^2 и RMSE (Root Mean Squared Error)
   - постройте диаграмму рассеяния для Length3 и Weight с линией регрессии.

Код программы (omo2_Auto_MPG.py):

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Настройки для графиков
sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)

def main():
    print("--- ЗАДАЧА 1: РЕГРЕССИЯ (Auto MPG) ---")

    # 1. Загрузка данных
    try:
        df_mpg = pd.read_csv('auto-mpg.csv', na_values='?')
    except FileNotFoundError:
        print("Ошибка: Файл 'auto-mpg.csv' не найден в текущей директории.")
        return

    # 2. Обработка пропусков (заполняем медианой)
    df_mpg['horsepower'] =
df_mpg['horsepower'].fillna(df_mpg['horsepower'].median())

    # Выбор признаков по заданию
    features = ['cylinders', 'horsepower', 'weight']
    target = 'mpg'

    X = df_mpg[features]
    y = df_mpg[target]

    # Разделение на обучающую и тестовую выборки
    X_train, X_test, y_train, y_test = train_test_split(
```

```
        X, y, test_size=0.2, random_state=42
    )

    # 3. Обучение модели линейной регрессии
    lin_reg = LinearRegression()
    lin_reg.fit(X_train, y_train)

    # Предсказание
    y_pred = lin_reg.predict(X_test)

    # 4. Расчет метрик
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    print(f"Метрика MSE: {mse:.2f}")
    print(f"Метрика R^2: {r2:.4f}")
    print(f"Коэффициенты модели: {dict(zip(features,
lin_reg.coef_.round(2)))}")
    print(f"Свободный член (intercept): {lin_reg.intercept_:.2f}")

    # 5. Визуализация
    plt.figure(figsize=(8, 5))
    sns.regplot(x='horsepower', y='mpg', data=df_mpg,
                scatter_kws={'alpha':0.5}, line_kws={'color':'red'})
    plt.title('Зависимость расхода топлива (mpg) от мощности (horsepower)')
    plt.xlabel('Мощность (horsepower)')
    plt.ylabel('Расход топлива (mpg)')
    plt.tight_layout()
    plt.show()

if __name__ == "__main__":
    main()
```

Код программы (omo2_Diabetes.py):

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, precision_score, recall_score,
confusion_matrix

from sklearn.preprocessing import StandardScaler


# Настройки для графиков

sns.set_theme(style="whitegrid")

plt.rcParams['figure.figsize'] = (10, 6)


def main():

    print("--- ЗАДАЧА 2: КЛАССИФИКАЦИЯ (Pima Indians Diabetes) ---")


    # 1. Загрузка данных

    col_names = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness',

                 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age',
'Outcome']


    try:

        df = pd.read_csv('pima-indians-diabetes.csv', comment='#',
header=None, names=col_names)

    except FileNotFoundError:

        print("Ошибка: Файл 'pima-indians-diabetes.csv' не найден в текущей
директории.")

        return


    # Разделение на признаки и целевую переменную

    X = df.drop('Outcome', axis=1)

    y = df['Outcome']


    # 2. Стандартизация признаков

    scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

# 3. Обучение модели логистической регрессии
log_reg = LogisticRegression(random_state=42)
log_reg.fit(X_train, y_train)

# Предсказание
y_pred = log_reg.predict(X_test)

# 4. Расчет метрик
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)

print(f"Accuracy (Общая точность): {accuracy:.4f}")
print(f"Precision (Точность): {precision:.4f}")
print(f"Recall (Полнота): {recall:.4f}")

# 5. Матрица ошибок и выводы
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Здоров (0)', 'Диабет (1)'],
            yticklabels=['Здоров (0)', 'Диабет (1)'])
plt.title('Матрица ошибок (Confusion Matrix)')
plt.xlabel('Предсказанный класс')
plt.ylabel('Истинный класс')
plt.tight_layout()
plt.show()
```

```
    # Анализ матрицы ошибок
    tn, fp, fn, tp = cm.ravel()
    print("\n--- Выводы по матрице ошибок ---")
    print(f"True Positives (TP - верно предсказан диабет): {tp}")
    print(f"True Negatives (TN - верно предсказано здоровье): {tn}")
    print(f"False Positives (FP - здорового назвали больным): {fp}")
    print(f"False Negatives (FN - больного назвали здоровым): {fn}")

    print("\nАнализ:")
    print("В медицинских задачах ложноотрицательные срабатывания (FN)
считаются наиболее опасными,")
    print("так как они означают, что модель пропустила болезнь у пациента.")
    print(f"В нашей модели FN = {fn}. Это означает, что {fn} больных людей
были ошибочно признаны здоровыми.")

if __name__ == "__main__":
    main()
```

Изображение результатов работы программы:

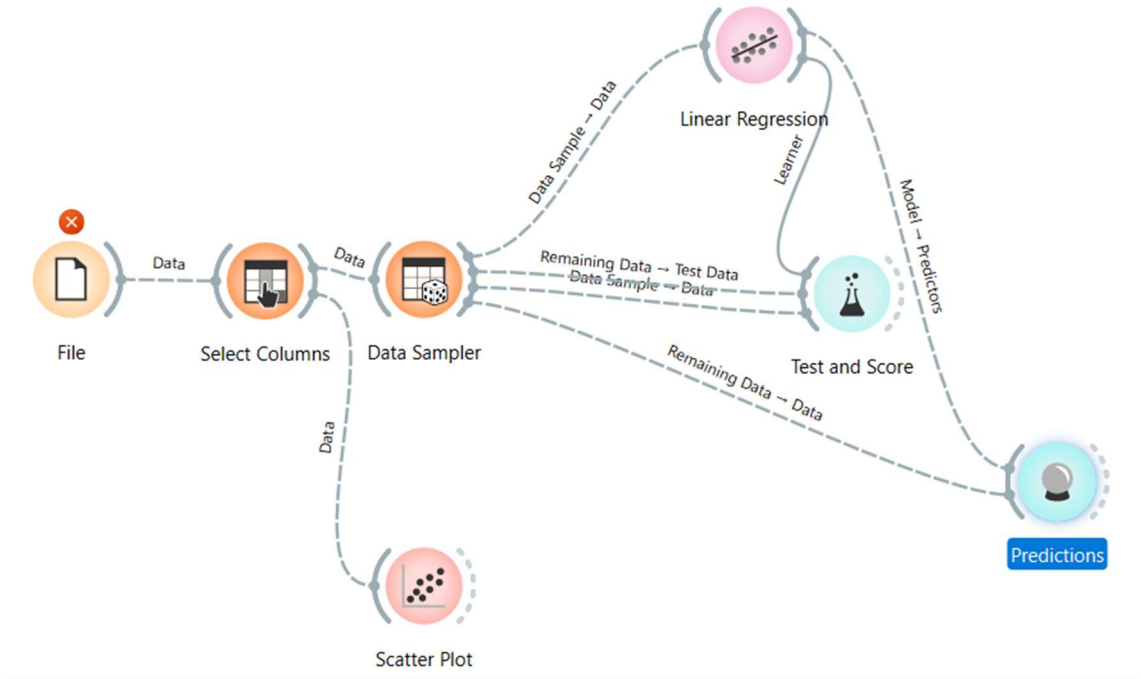


Рис. 1. Схема решение для задания 1 Регрессия (Прогнозирование расхода топлива) в программе Orange

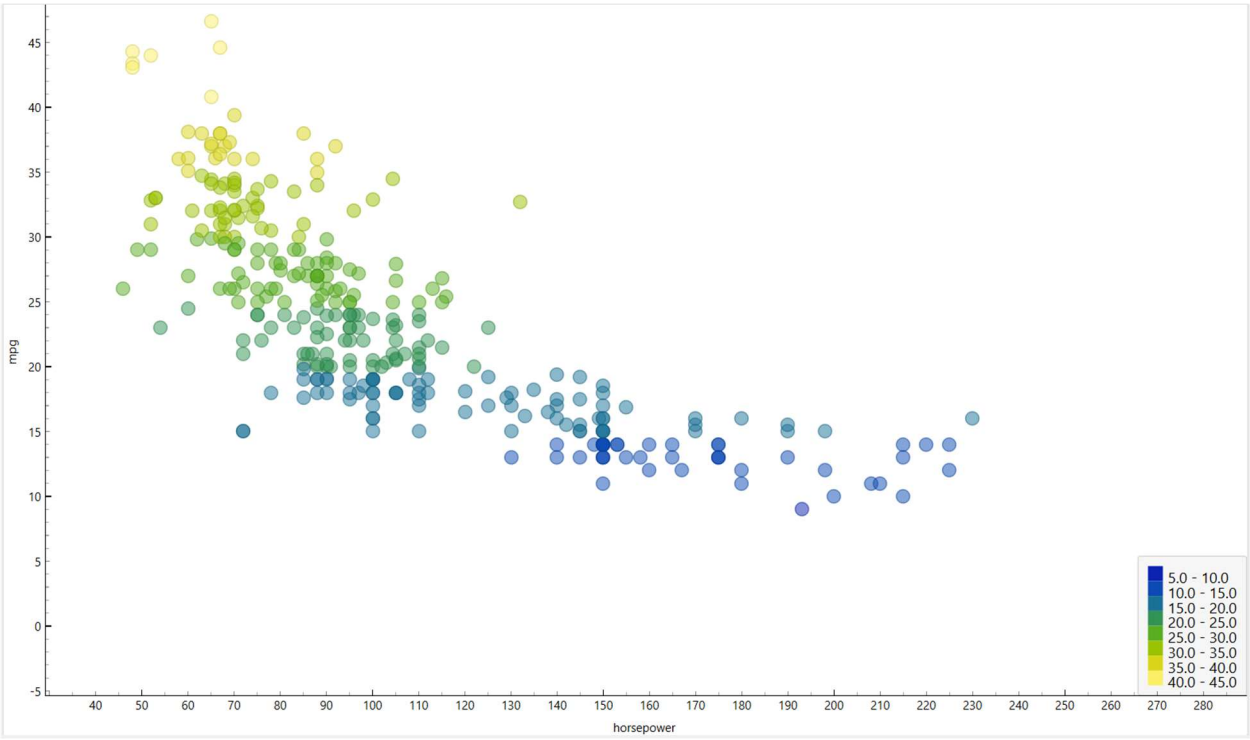


Рис. 2. Точечная диаграмма ‘mpg’ и ‘horsepower’ без линии регрессии в программе Orange

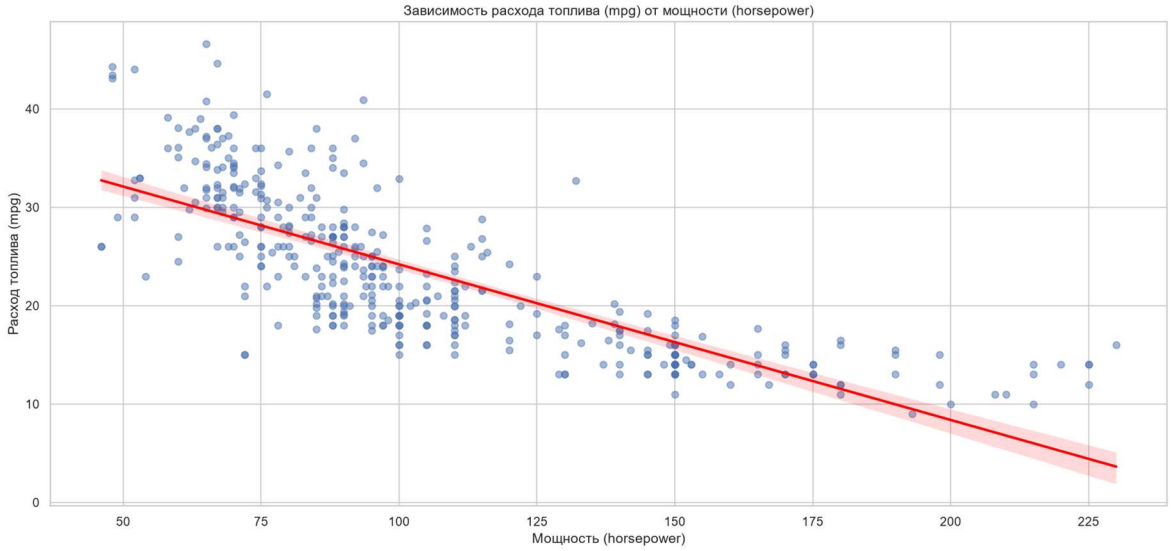


Рис. 3. Точечная диаграмма ‘mpg’ и ‘horsepower’ с линией регрессии в MathPlotLib

```
--- ЗАДАЧА 1: РЕГРЕССИЯ (Auto MPG) ---
Метрика MSE: 14.54
Метрика R^2: 0.7296
Коэффициенты модели: {'cylinders': np.float64(-0.37), 'horsepower': np.float64(-0.04), 'weight': np.float64(-0.01)}
Свободный член (intercept): 46.45
```

Рис. 4. Вывод программы в Python

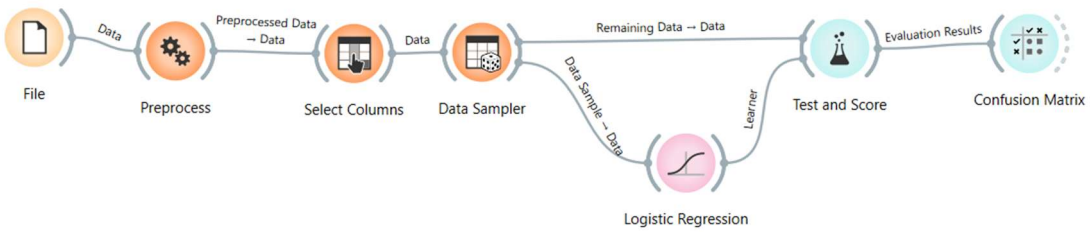


Рис. 5. Схема решения для задания 2 Классификация (Диагностика диабета) в программе Orange

Actual \ Predicted	0	1	Σ
0	81	14	95
1	26	32	58
Σ	107	46	153

Рис. 6. Матрица ошибок в программе Orange

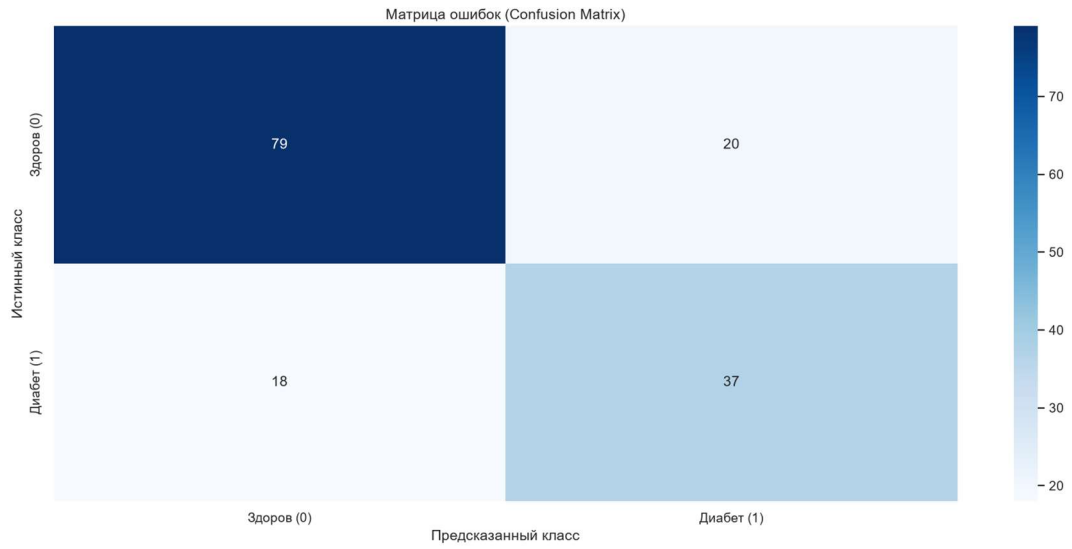


Рис. 7. Матрица ошибок в Python

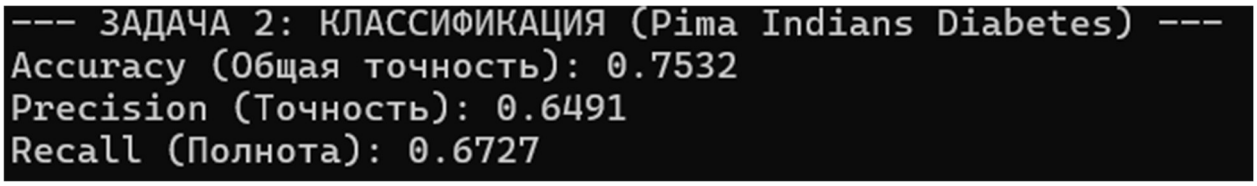


Рис. 8. Оценка модели логистической регрессии, созданной на Python

Вывод: изучил применение линейной и логистической регрессии для решения практических задач. Научился обучать модели, оценивать их качество с помощью соответствующих метрик и интерпретировать результаты.